

LAB # 8: To design the binary and signed binary divider using VHDL and reproduce output on FPGA board

Objectives

The objective of this lab is to *design* binary divider a signed binary number divider using VHDL

Part 1: Introduction, specification, design and testing of binary divider.

Part 2: Introduction, specification, design and testing of a signed binary number multiplier.

Part 1 – Introduction, specification, design and testing of a binary divider

We will consider the design of a parallel divider for positive binary numbers. As an example, we will design a circuit to divide an 8-bit dividend by a 4-bit divisor to obtain a 4-bit quotient. The following example illustrates the division process:

$$\begin{array}{r}
 \text{divisor } 1101 \overline{) 10000111} \\
 \underline{1101} \\
 0111 \\
 \underline{0000} \\
 1111 \\
 \underline{1101} \\
 0101 \\
 \underline{0000} \\
 0101 \text{ remainder}
 \end{array}$$

(135 ÷ 13 = 10 with a remainder of 5)

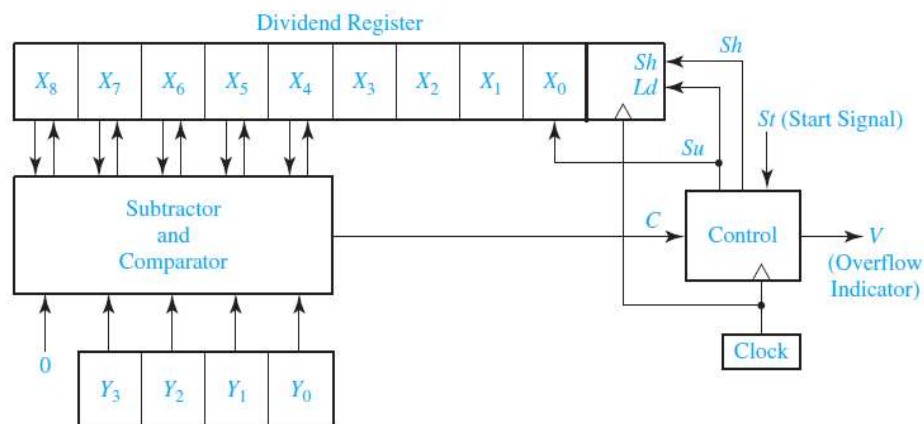
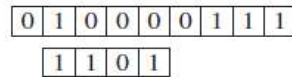
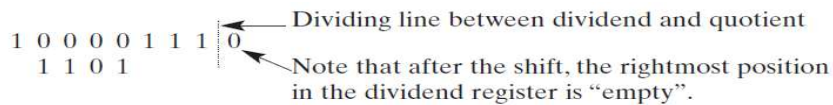


Figure 8.1 Block diagram of Binary divider

Just as binary multiplication can be carried out as a series of add and shift operations, division can be carried out by a series of subtraction and shift operations. To construct the divider, we will use a 9-bit dividend register and a 4-bit divisor register, as shown in Figure 8.1. During the division process, instead of shifting the divisor to the right before each subtraction as shown in the preceding example, we will shift the dividend to the left. Note that an extra bit is required on the left end of the dividend register so that a bit is not lost when the dividend is shifted left. Instead of using a separate register to store the quotient, we will enter the quotient bit-by-bit into the right end of the dividend register as the dividend is shifted left. Circuits for initially loading the dividend into the register will be added later. The preceding division example (135 divided by 13) is now reworked, showing the location of the bits in the registers at each clock time. Initially, the dividend and divisor are entered as follows:



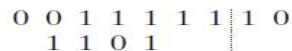
Subtraction cannot be carried out without a negative result, so we will shift before we subtract. Instead of shifting the divisor one place to the right, we will shift the dividend one place to the left:



Subtraction is now carried out, and the first quotient digit of 1 is stored in the unused position of the dividend register:



Next, we shift the dividend one place to the left:



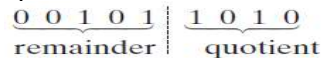
Because subtraction would yield a negative result, we shift the dividend to the left again, and the second quotient bit remains 0:



Subtraction is now carried out, and the third quotient digit of 1 is stored in the unused position of the dividend register:



A final shift is carried out and the fourth quotient bit is set to 0:



The final result agrees with that obtained in the first example. Note that in the first step the leftmost 1 in the dividend is shifted left into the leftmost position (X8) in the X register. If we did not have a place for this bit, the division operation would have failed at this step because $0000 < 1101$. However, by keeping the leftmost bit in X8, $10000 \geq 1101$, and subtraction can occur.

Task:

Draw the FSM diagram of binary divider controller circuit

1. Generate the required structural VHDL code, include it in your project, and compile the circuit.
2. Use functional simulation to verify that your code is correct

Part 2 – Introduction, specification, design and testing of a signed binary number divider

We now design a divider for signed (2's complement) binary numbers that divides a 32-bit dividend by a 16-bit divisor to give a 16-bit quotient. Although algorithms exist to divide the signed numbers directly, such algorithms are rather complex. So we take easy way out and complement the dividend and divisor if they are negative; when division is complete, we complement the quotient if it should be negative.

Figure 8.2 shows a block diagram for the divider. We use a 16 bit bus to load the registers. Since the dividend is 32 bits, two clocks are required to load the upper and lower halves of the dividend register, and one clock is needed to load the divisor. An extra sign flip flop is used to store the sign of the dividend. We will use a dividend register with a built-in 2's complemener. The subtractor consists of an adder and a complemener, so subtraction can be accomplished by adding 2's complement of the divisor to the dividend register. If the divisor is negative, using a separate step to complement it is unnecessary; we can simply disable the complemener and add the negative divisor instead of subtracting its complement. The control network is divided into two parts – a main control, which determines the sequence of shifts and subtracts, and a counter, which counts the number of shifts. The counter counts a signal K=1 when 15 shifts have occurred. Control signals are defined as follows:

LdU	Load upper half of dividend from bus
LdL	Load lower half of dividend from bus
Lds	Load sign of dividend into sign flip flop
S	Sign of dividend
CmL	complement dividend register (2's complement)
Ldd	Load divisor from bus
Su	Enable adder output onto bus (Ena) and load upper half of dividend from bus
Cm2	Enable complemener. (Cm2 equals the complement of the sign bit of the divisor, so a positive divisor is complemented and a negative divisor is not.)
Sh	Shift the dividend register left one place and increment the counter
C	Carry output from adder. If C=1 the divisor can be subtracted from the upper dividend
St	Start
V	Overflow

Qneg Quotient will be negative. (Qneg=1 when the sign of the dividend and divisor are different)

Procedure for carrying out the signed division is as follows:

1. Load the upper half of the dividend from the bus, and copy the sign of the dividend in to the sign flip flop.
2. Load the lower half of the dividend from the bus
3. Load the divisor from the bus
4. Complement the dividend if it is negative
5. If an overflow condition is present, go to the done state.
6. Else carry out the division by a series of shifts and subtracts
7. When division is complete, complement the quotient if necessary, and go to the done state.

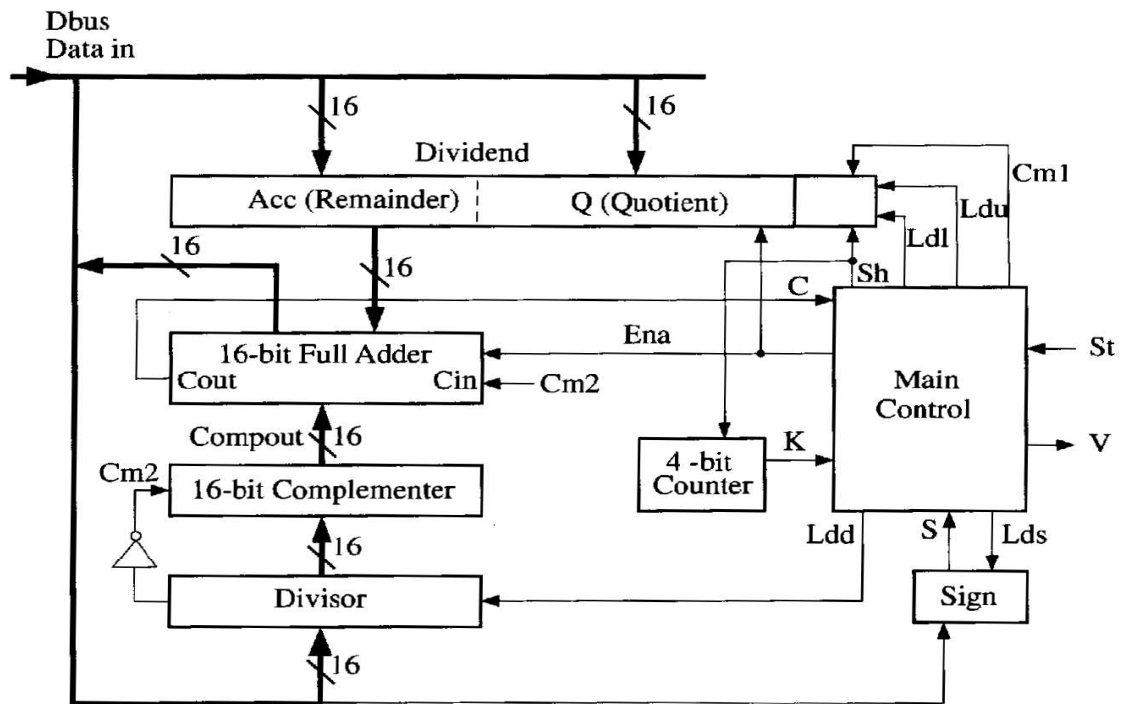


Figure 8.2 Block diagram of Signed divider

Draw the FSM diagram of the controller

1. Generate the required structural VHDL code, include it in your project, and compile the circuit.
2. Use functional simulation to verify that your code is correct.

Practice Tasks: Transfer VHDL code into the FPGA board

This exercise concerns the design of a divider for unsigned binary numbers that will divide a 16-bit dividend by an 8-bit divisor to give an 8-bit quotient. Assume that the start signal (ST=1) is 1 for exactly one clock cycle time. If the quotient would require more than 8 bits, the divider should stop immediately and output V=1 to indicate an overflow. Use a 17-bit dividend register and store the quotient in the lower 8 bits of this register. Use a 4-bit counter to count the number of shifts, together with a subtract-shift controller.

- a) Draw the block diagram of the divider.
- b) Draw the state graph for the subtract-shift controller (3 states)
- c) Write a VHDL description of the divider.